

Redux Toolkit & Redux Persist – Complete Guide with Examples

This document explains Redux Toolkit and Redux Persist in a simple, practical way with real examples. It is suitable for beginners and working professionals.

1. What is Redux Toolkit?

Redux Toolkit (RTK) is the official, recommended way to write Redux logic. It reduces boilerplate code and provides best practices by default.

- Simplifies store configuration
- Uses Immer internally (allows mutable-looking code)
- Includes createSlice, configureStore, createAsyncThunk

2. What is Redux Persist?

Redux Persist allows you to save Redux state to persistent storage (like localStorage) so that state is not lost on page refresh.

- Keeps user logged in after refresh
- Persists selected slices only

3. When to use Redux Persist?

- Authentication data (token, user info)
- User preferences (theme, language)
- Cart data in e-commerce apps

4. Basic Redux Toolkit Setup

Auth Slice Example:

```
// features/auth/authSlice.js
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
  user: null,
  token: null,
  isAuthenticated: false,
};

const authSlice = createSlice({
  name: "auth",
  initialState,
  reducers: {
    loginSuccess(state, action) {
```

```

        state.user = action.payload.user;
        state.token = action.payload.token;
        state.isAuthenticated = true;
    },
    logout(state) {
        state.user = null;
        state.token = null;
        state.isAuthenticated = false;
    },
},
});

export const { loginSuccess, logout } = authSlice.actions;
export default authSlice.reducer;

```

5. Redux Persist Setup

```

// store/store.js
import { configureStore, combineReducers } from "@reduxjs/toolkit";
import authReducer from "../features/auth/authSlice";
import storage from "redux-persist/lib/storage";
import { persistReducer, persistStore } from "redux-persist";

const persistConfig = {
    key: "root",
    storage,
    whitelist: ["auth"], // only auth will persist
};

const rootReducer = combineReducers({
    auth: authReducer,
});

const persistedReducer = persistReducer(persistConfig, rootReducer);

export const store = configureStore({
    reducer: persistedReducer,
});

export const persistor = persistStore(store);

```

6. PersistGate Usage

```

// index.js
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import { PersistGate } from "redux-persist/integration/react";
import App from "../App";
import { store, persistor } from "../store/store";

ReactDOM.createRoot(document.getElementById("root")).render(
    <Provider store={store}>
        <PersistGate loading={null} persistor={persistor}>

```

```
    <App />
  </PersistGate>
</Provider>
);
```

7. How It Works (Flow)

- User logs in → auth slice updated
- Redux Persist saves auth state to localStorage
- Page refresh → state rehydrated automatically

8. Common Mistakes

- Persisting entire store unnecessarily
- Storing sensitive data without encryption
- Forgetting PersistGate

9. Summary

Redux Toolkit simplifies Redux development, while Redux Persist ensures important state survives page reloads. Together, they provide a clean, scalable state management solution.